The continued improvements to Ruby guarantee relevance for Ruby developers. Ruby isn't going anywhere, and it isn't a goal of this book to try to replace Ruby with Elixir.

Next, let's explore a bit of what makes Elixir special.

The Case for Elixir

Anytime you pick a new technology, you'll have to make a case for it. The first consideration is your personal decision of whether you want to spend time learning the language. Next, you'll need to bring it to your professional peers and get their buy-in that they also want to spend time learning it. Finally, your business will need to make the case of whether they want to invest in it. Even if you're just a hobbyist who is curious about Elixir, you still need to justify learning it over other languages.

The technical aspects of a language are obviously important in these decisions, but the nontechnical aspects are just as important. In the previous section, we spent a lot of time talking about the nontechnical aspects that make Ruby a joy to use and learn.

We'll briefly cover a few nontechnical strengths of Elixir, but we'll also look at why the technology itself is appealing.

Nontechnical Strengths

Elixir is small and relatively new. These are two things that you'll be working against when you make the case for it. However, Elixir finds strength in its community, innovation, and philosophy.

Close-Knit Community

The Elixir Forums[4] and Elixir Slack[5] are a wealth of knowledge and friendly faces. It's rare that a question goes unanswered in these places because the community shows up to help.

Culture of Testing

Elixir, like Ruby, has a healthy testing culture that permeates every layer of the technical stack. It's rare to find untested libraries.

Language Design Philosophy

Elixir doesn't have a vocalized philosophy like Ruby does, but it's clear that it's influenced by other languages. Elixir's creator, José Valim[6], has

4. https://elixirforum.com
5. https://elixir-slack.community/
6. https://twitter.com/josevalim